Computer Vision (Practical)

Practical File – BCA Hons., Semester 7

Practical 1: Image Read, Display & Basic Operations

Aim:

To load, display, resize, crop, rotate and save an image using OpenCV.

Code:

```
import cv2
import matplotlib.pyplot as plt

# Load the image (replace with your own file path)
img = cv2.imread('image.jpg')

# Display original image
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Original Image'); plt.axis('off'); plt.show()

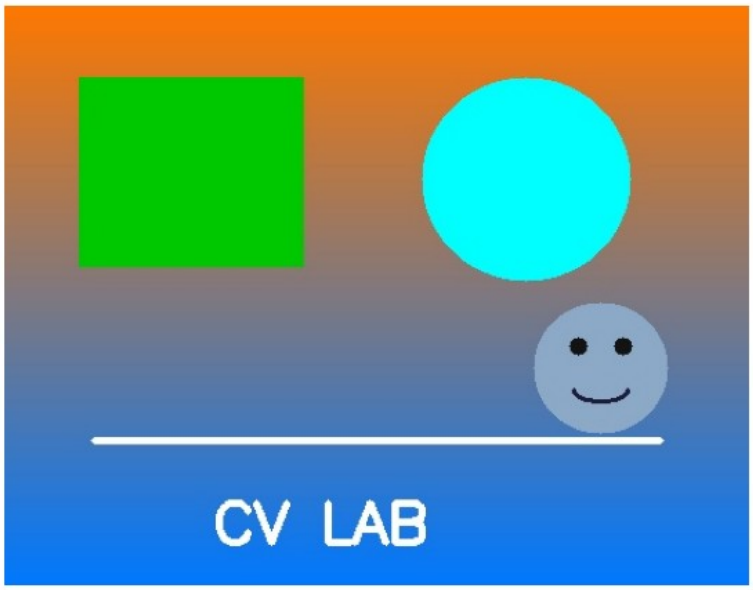
# Resize
resized = cv2.resize(img, (200, 200))
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title('Resized'); plt.axis('off'); plt.show()

# Crop
h, w = img.shape[:2]
cropped = img[50:h-50, 50:w-50]
plt.imshow(cv2.cvtColor(cropped, cv2.COLOR_BGR2RGB))
plt.title('Cropped'); plt.axis('off'); plt.show()

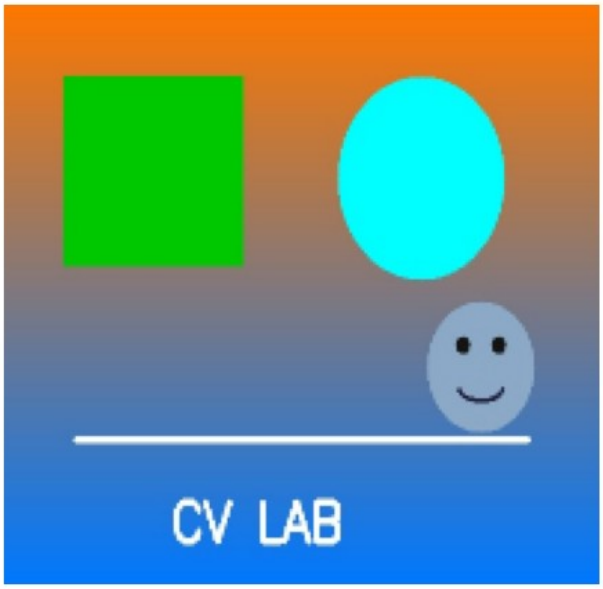
# Rotate by 45 degrees
M = cv2.getRotationMatrix2D((w/2, h/2), 45, 1)
rotated = cv2.warpAffine(img, M, (w, h))
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title('Rotated'); plt.axis('off'); plt.show()

# Save the image to disk
cv2.imwrite('saved_image.jpg', img)
```

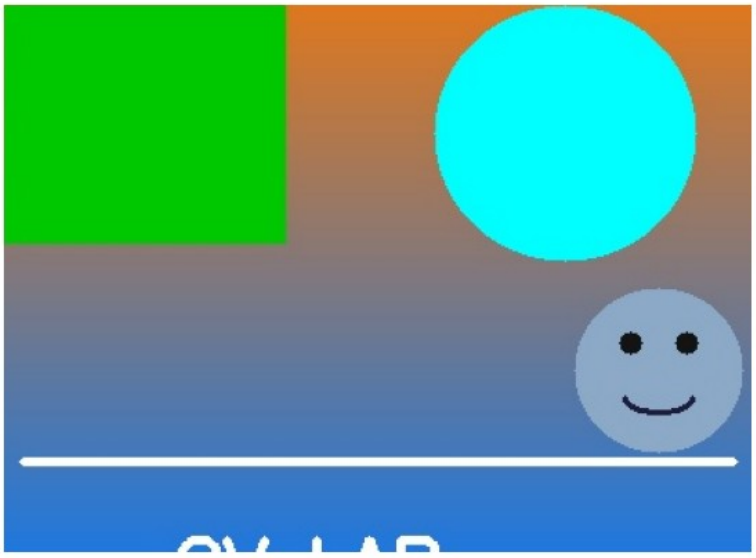
Output:



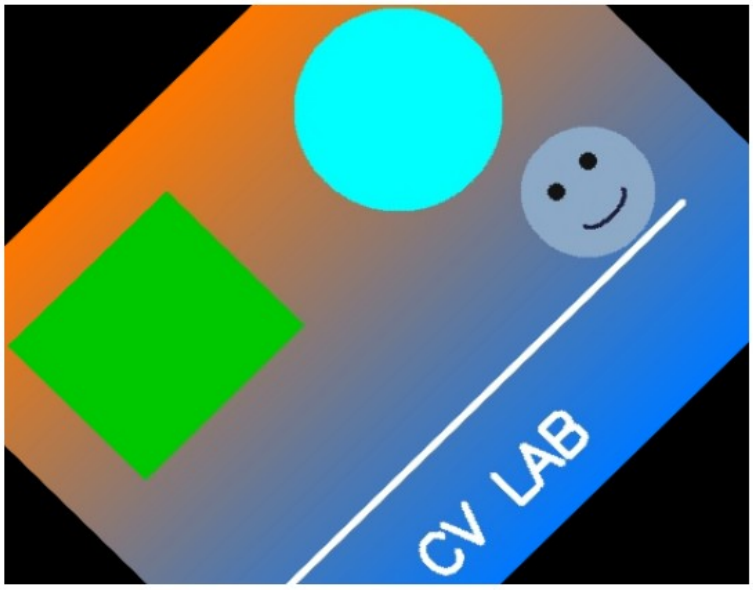
Original image



Resized (200x200)



Cropped



Rotated 45°

Practical 2: Color Space Conversion

Aim:

To convert an image between RGB, Grayscale and HSV color spaces.

Code:

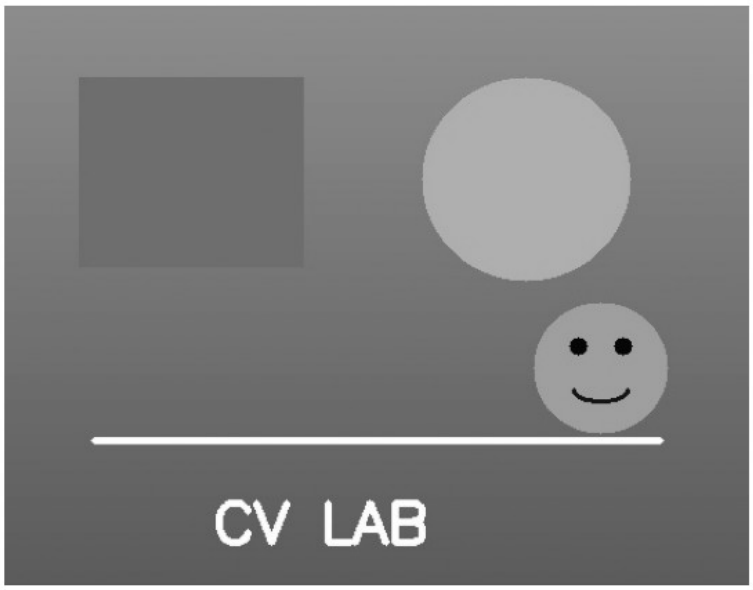
```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')

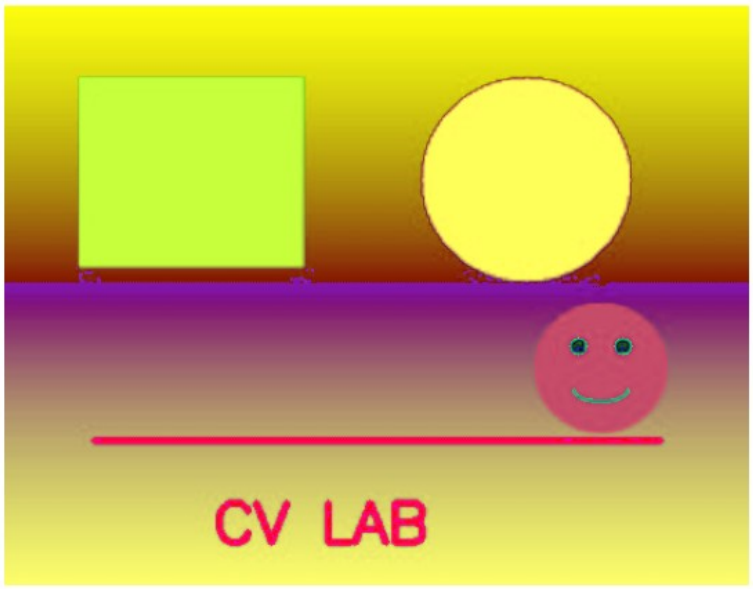
# Convert to Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(gray, cmap='gray')
plt.title('Grayscale'); plt.axis('off'); plt.show()

# Convert to HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
plt.imshow(cv2.cvtColor(hsv, cv2.COLOR_BGR2RGB))
plt.title('HSV'); plt.axis('off'); plt.show()
```

Output:



Grayscale



HSV

Practical 3: Image Enhancement Techniques

Aim:

To adjust brightness/contrast and apply Gaussian and Median smoothing filters.

Code:

```
import cv2
import matplotlib.pyplot as plt

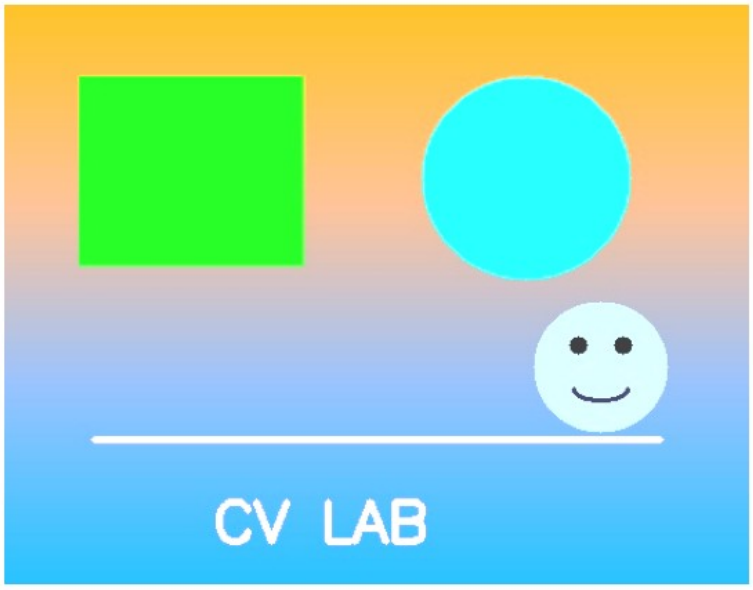
img = cv2.imread('image.jpg')

# Brightness / contrast adjustment
# alpha = contrast control, beta = brightness control
bright = cv2.convertScaleAbs(img, alpha=1.3, beta=40)
plt.imshow(cv2.cvtColor(bright, cv2.COLOR_BGR2RGB))
plt.title('Brightness/Contrast Adjusted'); plt.axis('off'); plt.show()

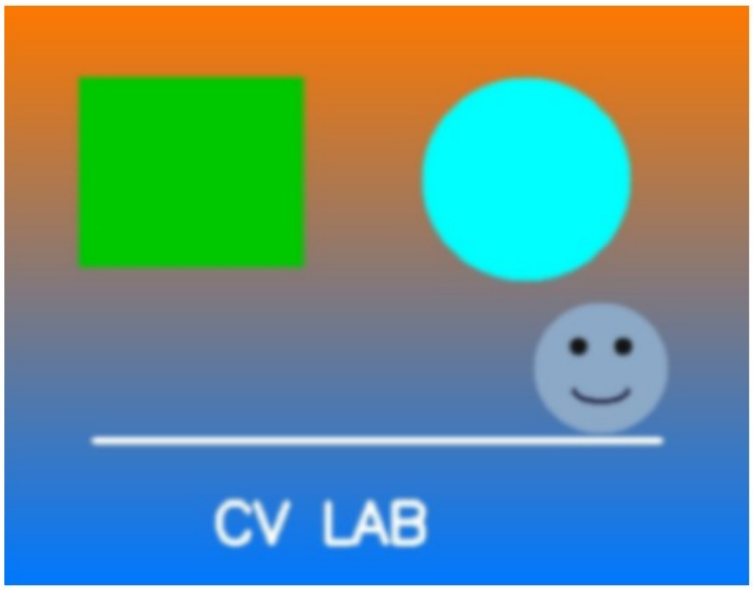
# Gaussian smoothing
gaussian = cv2.GaussianBlur(img, (9, 9), 0)
plt.imshow(cv2.cvtColor(gaussian, cv2.COLOR_BGR2RGB))
plt.title('Gaussian Blur'); plt.axis('off'); plt.show()

# Median smoothing
median = cv2.medianBlur(img, 9)
plt.imshow(cv2.cvtColor(median, cv2.COLOR_BGR2RGB))
plt.title('Median Blur'); plt.axis('off'); plt.show()
```

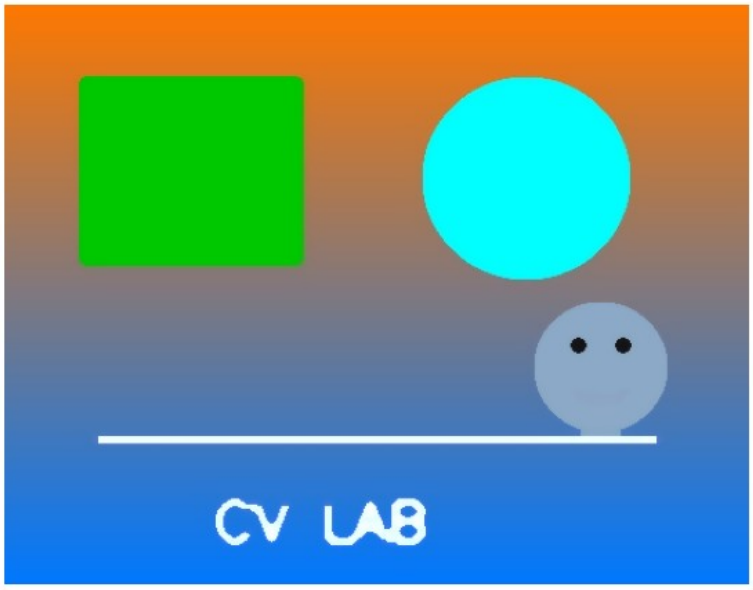
Output:



Brightness / Contrast adjusted



Gaussian blur



Median blur

Practical 4: Histogram Processing

Aim:

To plot the histogram of an image and perform histogram equalization.

Code:

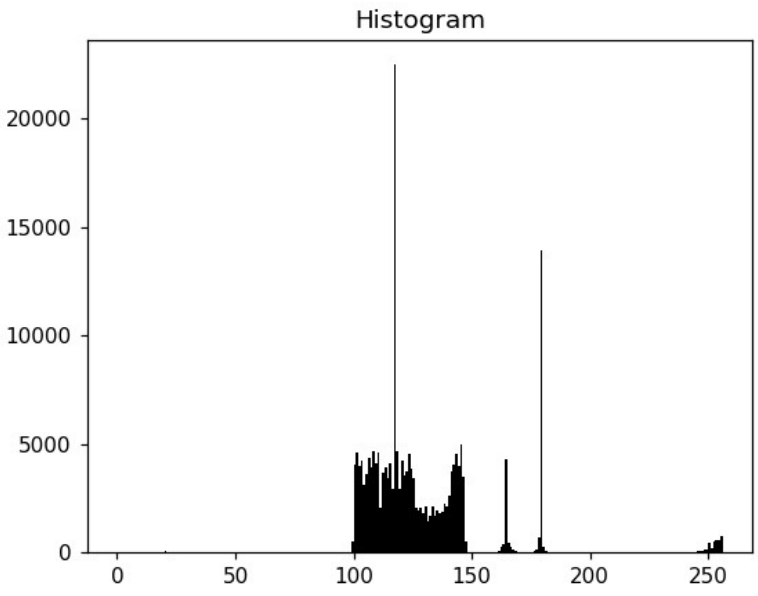
```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

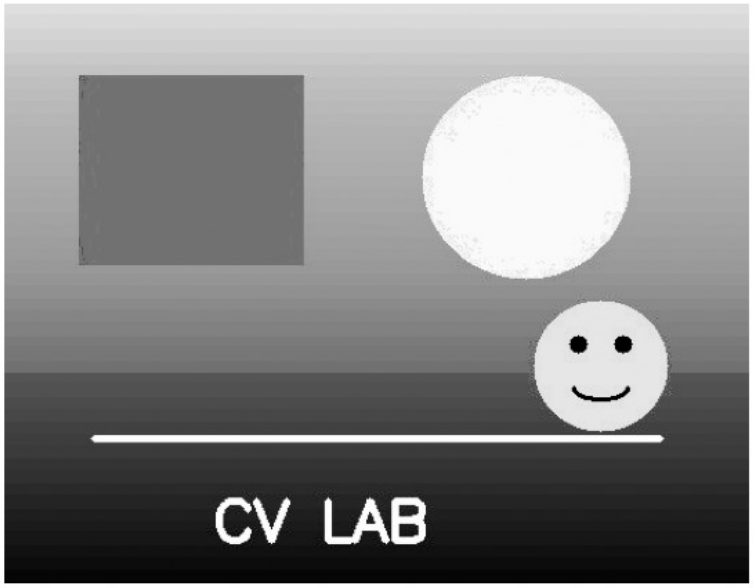
# Plot histogram
plt.hist(gray.ravel(), bins=256, range=[0, 256], color='black')
plt.title('Histogram'); plt.xlabel('Pixel Value'); plt.ylabel('Frequency')
plt.show()

# Histogram equalization
equalized = cv2.equalizeHist(gray)
plt.imshow(equalized, cmap='gray')
plt.title('Histogram Equalized'); plt.axis('off'); plt.show()
```

Output:



Histogram



Histogram equalized

Practical 5: Edge Detection

Aim:

To detect edges in an image using the Sobel and Canny operators.

Code:

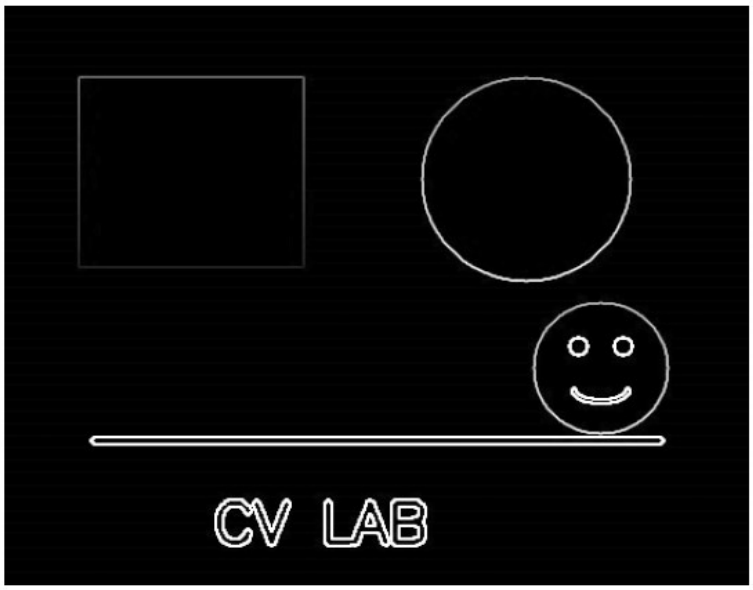
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

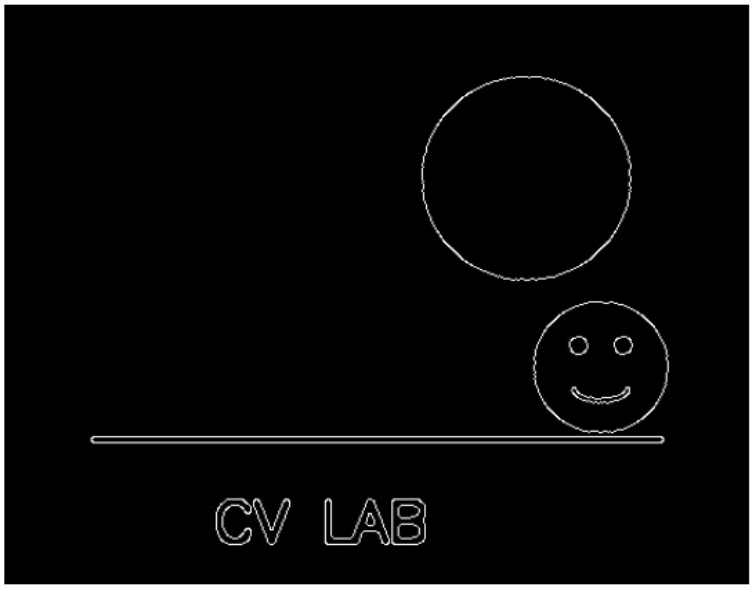
# Sobel edge detection
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel = cv2.magnitude(sx, sy)
sobel = np.uint8(np.clip(sobel, 0, 255))
plt.imshow(sobel, cmap='gray')
plt.title('Sobel Edge Detection'); plt.axis('off'); plt.show()

# Canny edge detection
canny = cv2.Canny(gray, 100, 200)
plt.imshow(canny, cmap='gray')
plt.title('Canny Edge Detection'); plt.axis('off'); plt.show()
```

Output:



Sobel edge detection



Canny edge detection

Practical 6: Corner Detection

Aim:

To detect corners in an image using the Harris Corner Detector.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

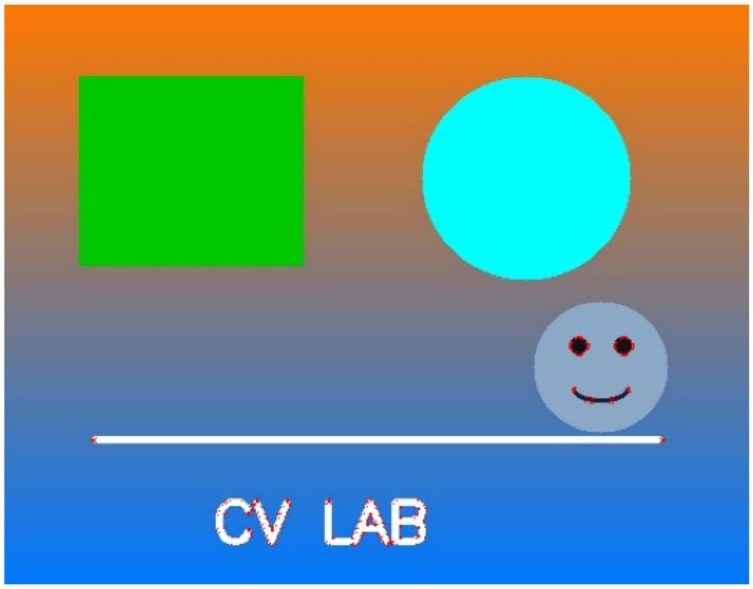
img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_f = np.float32(gray)

# Harris corner detection
harris = cv2.cornerHarris(gray_f, blockSize=2, ksize=3, k=0.04)
harris = cv2.dilate(harris, None)   # mark corners more clearly

out = img.copy()
out[harris > 0.01 * harris.max()] = [0, 0, 255]   # mark corners in red

plt.imshow(cv2.cvtColor(out, cv2.COLOR_BGR2RGB))
plt.title('Harris Corner Detection'); plt.axis('off'); plt.show()
```

Output:



Harris corners marked in red

Practical 7: Feature Detection & Matching

Aim:

To detect and match keypoints between two images using ORB.

Code:

```
import cv2
import matplotlib.pyplot as plt

# Load two images to compare/match
img1 = cv2.imread('image1.jpg')
img2 = cv2.imread('image2.jpg')

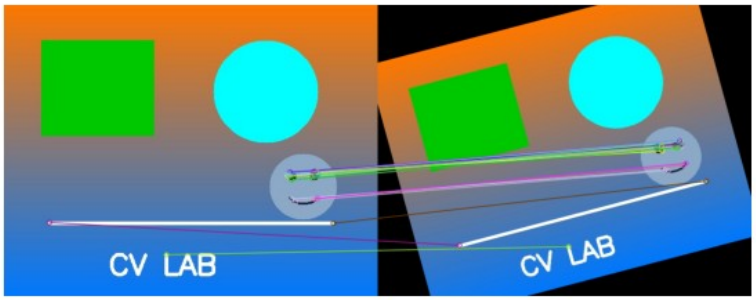
# Create ORB detector
orb = cv2.ORB_create()

# Detect keypoints and descriptors
kp1, des1 = orb.detectAndCompute(img1, None)
kp2, des2 = orb.detectAndCompute(img2, None)

# Match descriptors using Brute-Force Hamming matcher
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda x: x.distance)

# Draw top 20 matches
matched_img = cv2.drawMatches(img1, kp1, img2, kp2, matches[:20], None, flags=2)
plt.figure(figsize=(10,5))
plt.imshow(cv2.cvtColor(matched_img, cv2.COLOR_BGR2RGB))
plt.title('ORB Feature Matching'); plt.axis('off'); plt.show()
```

Output:



ORB keypoint matches between original and a rotated/scaled copy

Practical 8: Image Segmentation

Aim:

To segment an image using binary thresholding, adaptive thresholding and K-means clustering.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

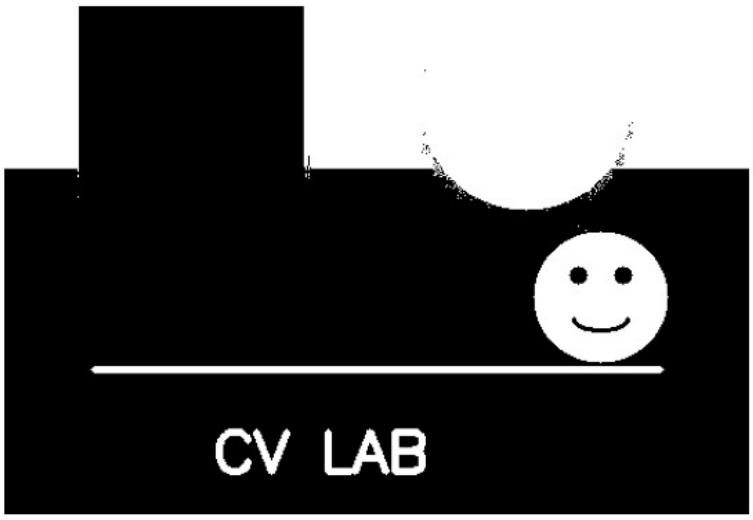
img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Binary thresholding
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
plt.imshow(binary, cmap='gray')
plt.title('Binary Thresholding'); plt.axis('off'); plt.show()

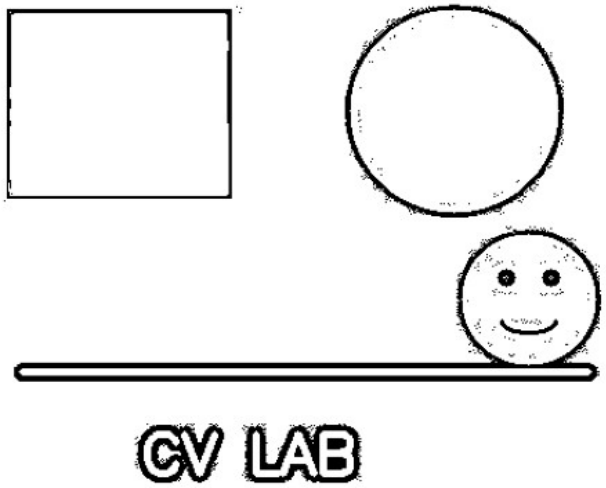
# Adaptive thresholding
adaptive = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                 cv2.THRESH_BINARY, 11, 2)
plt.imshow(adaptive, cmap='gray')
plt.title('Adaptive Thresholding'); plt.axis('off'); plt.show()

# K-means clustering segmentation
Z = img.reshape((-1, 3)).astype(np.float32)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
K = 3
_, labels, centers = cv2.kmeans(Z, K, None, criteria, 10,
cv2.KMEANS_RANDOM_CENTERS)
centers = np.uint8(centers)
segmented = centers[labels.flatten()].reshape(img.shape)
plt.imshow(cv2.cvtColor(segmented, cv2.COLOR_BGR2RGB))
plt.title('K-means Segmentation'); plt.axis('off'); plt.show()
```

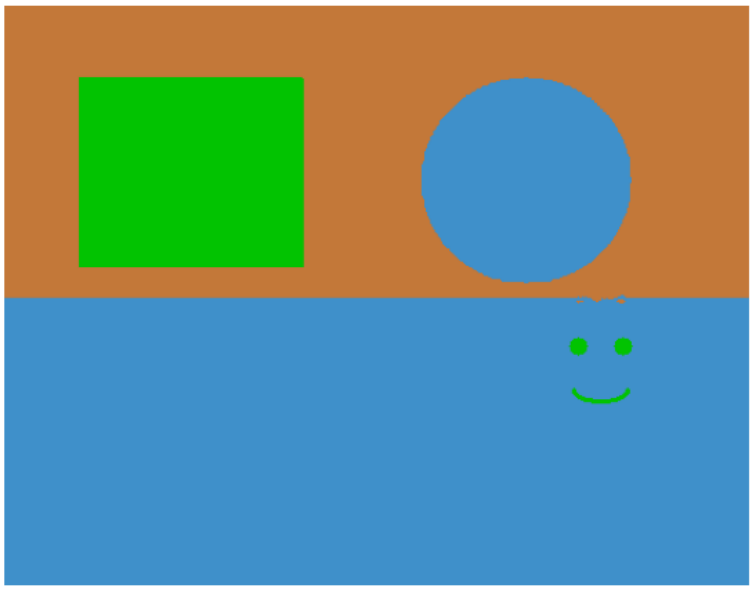
Output:



Binary thresholding



Adaptive thresholding



K-means segmentation (K=3)

Practical 9: Face Detection

Aim:

To detect faces in an image using the Haar Cascade Classifier.

Code:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

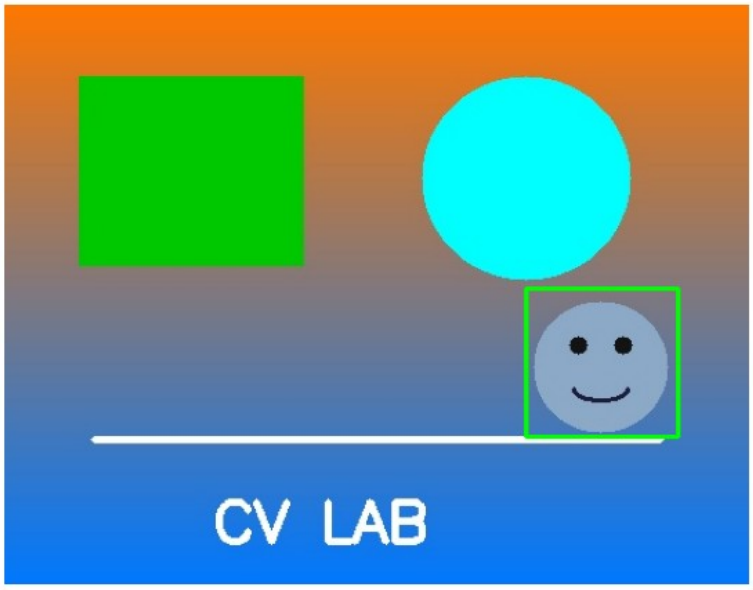
# Load OpenCV's pretrained Haar Cascade face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

# Detect faces
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)

# Draw rectangles around detected faces
out = img.copy()
for (x, y, w, h) in faces:
    cv2.rectangle(out, (x, y), (x+w, y+h), (0, 255, 0), 2)

plt.imshow(cv2.cvtColor(out, cv2.COLOR_BGR2RGB))
plt.title(f'Faces Detected: {len(faces)}'); plt.axis('off'); plt.show()
```

Output:



Detected face region marked with a green rectangle

Practical 10: Motion Detection / Object Tracking

Aim:

To detect motion between video frames using background subtraction.

Code:

```
import cv2

# 0 = default webcam; replace with a video file path if needed
cap = cv2.VideoCapture(0)

# Background subtractor object (learns the static background over frames)
back_sub = cv2.createBackgroundSubtractorMOG2()

while True:
    ret, frame = cap.read()
    if not ret:
        break

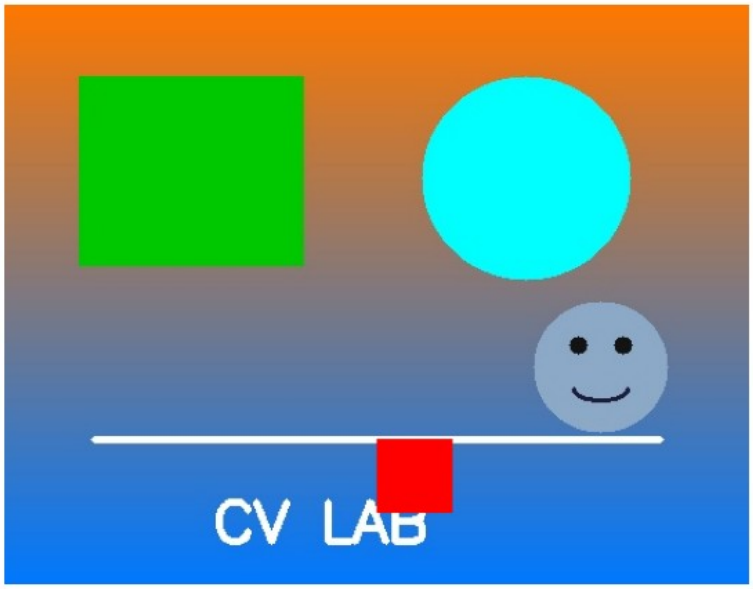
    # Apply background subtraction to get the foreground (moving) mask
    fg_mask = back_sub.apply(frame)

    cv2.imshow('Original', frame)
    cv2.imshow('Foreground Mask', fg_mask)

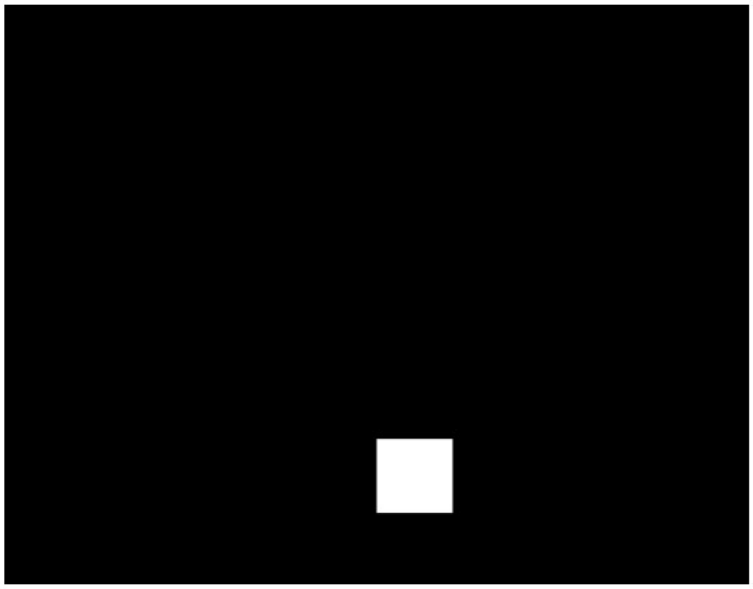
    # Press 'q' to quit
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Output:



Current frame (new object appears)



Foreground mask (moving/new region highlighted)